2) Programming / Computers

Why start at 0? Why not just set the starting point in computer memory to 1?

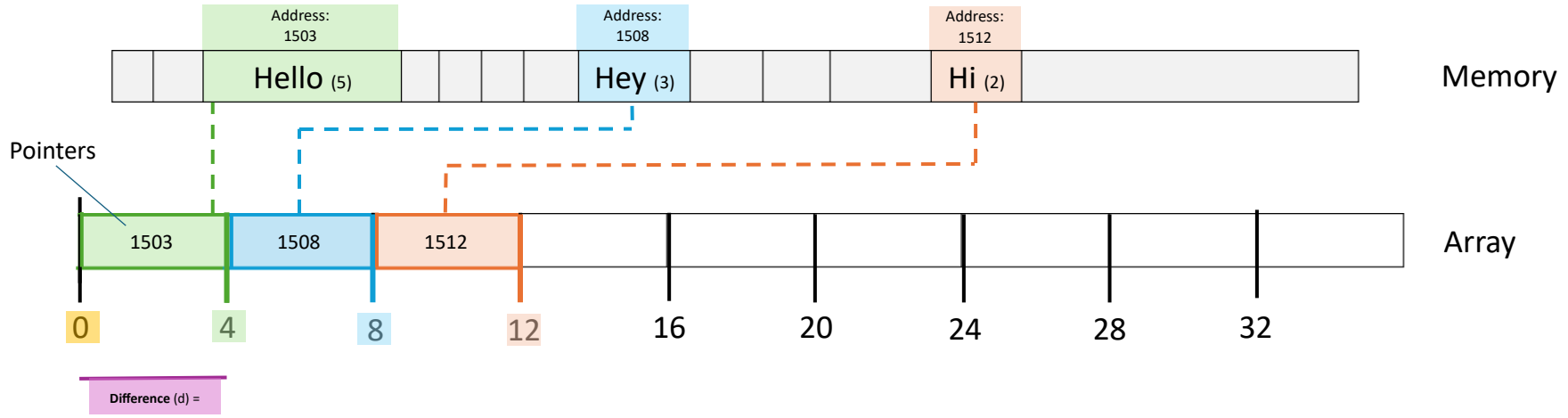
This, too, has to do with mathematics — specifically, the formula for arithmetic progressions mentioned earlier.

The computer also needs to retrieve the requested element from an array as quickly as possible (i.e., efficiently and with high performance).

a) String Arrays

array = { "Hello", "Hey", "Hi" }

The computer stores this as follows:



Since, for example, in a string array each element has or can have a different length (variable length), the array does not store the value itself, but rather the address of the value in memory (= pointer). Since this address has a fixed length, the formula for arithmetic progressions can then be easily applied to determine the memory address (pointer) of the value — and thus the value itself — for a specific or desired index.

To retrieve a specific element from the array (as we recall a_n), we do the following:

array[i]

Since this is 0-based, the element can be retrieved by the computer very easily and with excellent performance using the formula for arithmetic progressions:

array[2]

$a_n = a_0 + n \cdot d$

$a_0 = 0$
$n = 2$
$d = 4$

*n = i

$a_2 = 0 + 2 \cdot 4$

$a_2 = 8$

8 corresponds to the memory address where the memory address of the element being searched for (= pointer) is stored. In this case, memory address 8 points to the value 1508, which is the memory address of the element being searched for. Based on this, the correct value for i = 2 can now be returned — in this case, “Hey” — and this with extremely high performance.

Of course, to make things easier for humans, one could use “1” instead of “0” in syntactically. However, as discussed in the section on mathematics, this would require subtracting 1 somewhere during every access to maintain the mathematical advantage. With a very high number of accesses, this would no longer be as performant and efficient. This was especially true in the early days of computers (i.e. limited processing power).

Excursus:
For a computer, it is actually (slightly) more computationally intensive to calculate a subtraction than an addition — or, of course, nothing at all (see two’s complement). This is because a computer can only perform addition, so it must flip every bit (since negative numbers are stored this way) and then add the result.

Example:

5 - 3

3 in binary	=	0011
This becomes:		1100

5 + (-3) = 2

That is obviously slower and less efficient than a simple addition — or of course no operation at all.

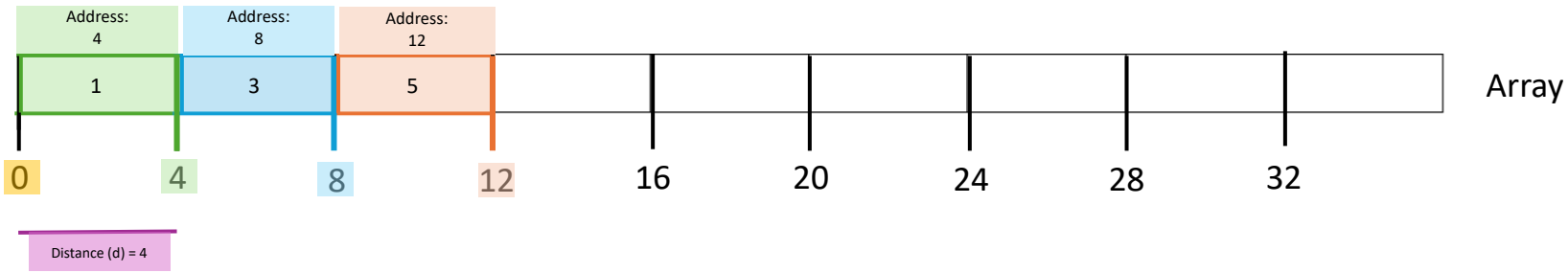
This was particularly true in the early days of computing, when hardware was expensive and slow.

b) Number Arrays

With numeric arrays, things are a bit simpler, since numbers each have a fixed length (e.g., 4 bits) in memory. They are therefore stored directly in the array.

array = { 1, 3, 5 }

The computer stores this as follows:



To retrieve a specific element from the array, we can — since numbers, as mentioned, have a fixed length in memory — simply apply the formula from mathematics or arithmetic progressions:

array[2]

$a_n = a_0 + n \cdot d$

$a_0 = 0$
$n = 2$
$d = 4$

*n = i

$a_2 = 0 + 2 \cdot 4$

$a_2 = 8$

8 corresponds to the memory address of the element or value being searched for — in this case, 3. This can now be retrieved extremely quickly.